

Київський національний університет імені Тараса Шевченка



ЗВІТ

до лабораторної роботи №6

З дисципліни БАЗИ ДАНИХ ТА ІНФОРМАЦІЙНІ СИСТЕМИ

Виконав
Студент 3 курсу
Групи МІТ-31
Факультету Інформаційних Технологій
Лаптев Олександр

Київ, 2026

Тема: Поглиблене вивчення MongoDB: оптимізація продуктивності, використання шардінгу та реплікації

Мета: Ознайомитися з методами підвищення продуктивності MongoDB. Навчитися використовувати індекси для оптимізації запитів. Дослідити механізми реплікації та шардінгу.

Короткий теоретичний опис основних концепцій

- **Індекси:** Спеціальні структури даних у MongoDB, які дозволяють прискорити пошук та вибірку документів, зменшуючи кількість сканованих записів (уникаючи Collection Scan). Водночас індекси можуть сповільнювати операції запису через необхідність їх оновлення.
- **Реплікація (Replica Set):** Механізм синхронізації даних між кількома серверами для забезпечення відмовостійкості та високої доступності. Кластер складається з одного головного вузла (Primary) для запису/читання та кількох підлеглих (Secondary), які дублюють дані.
- **Шардінг:** Метод горизонтального масштабування, при якому великі обсяги даних розподіляються між кількома незалежними серверами (шардами) за певним ключем (Shard Key), що дозволяє балансувати навантаження.

Хід роботи :

Частина 1: Оптимізація продуктивності запитів

1. Створення локальної бази MongoDB

Встановити MongoDB та розширення MongoDB for VS Code.

Підключитися до MongoDB через MongoDB Explorer.

Створити базу `performance_test` та колекцію `sales`.

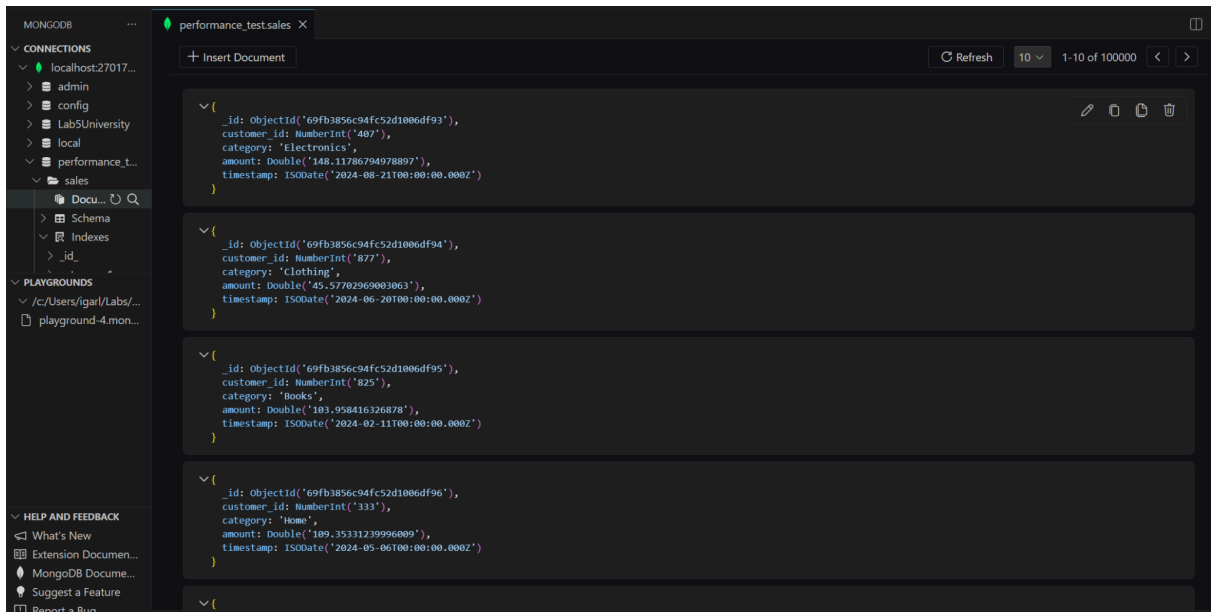


Рис 1. Створення колекції в VS Code

2. Генерація та вставка тестових даних

Написати Python-скрипт `insert_data.py` для генерації 100 000 документів.

Виконати скрипт та переконаватися, що дані вставлено.

```
import random
import datetime
from pymongo import MongoClient

# Підключення до MongoDB
client = MongoClient("mongodb://localhost:27017")
db = client["performance_test"]
collection = db["sales"]

# Створення тестових даних
categories = ["Electronics", "Clothing", "Books", "Home", "Sports"]

documents = [
    {
        "customer_id": random.randint(1, 1000),
        "category": random.choice(categories),
        "amount": random.uniform(5, 500),
        "timestamp": datetime.datetime(2024, random.randint(1, 12),
random.randint(1, 28))
    }
]
```

```
        for _ in range(100000)
    ]

    # Вставка даних у MongoDB
    print("Починаю вставку 100 000 документів...")
    collection.insert_many(documents)
    print("Дані успішно вставлено!")
```

Рис. 2. Код з файлу insert_data , що виконує генерацію документів

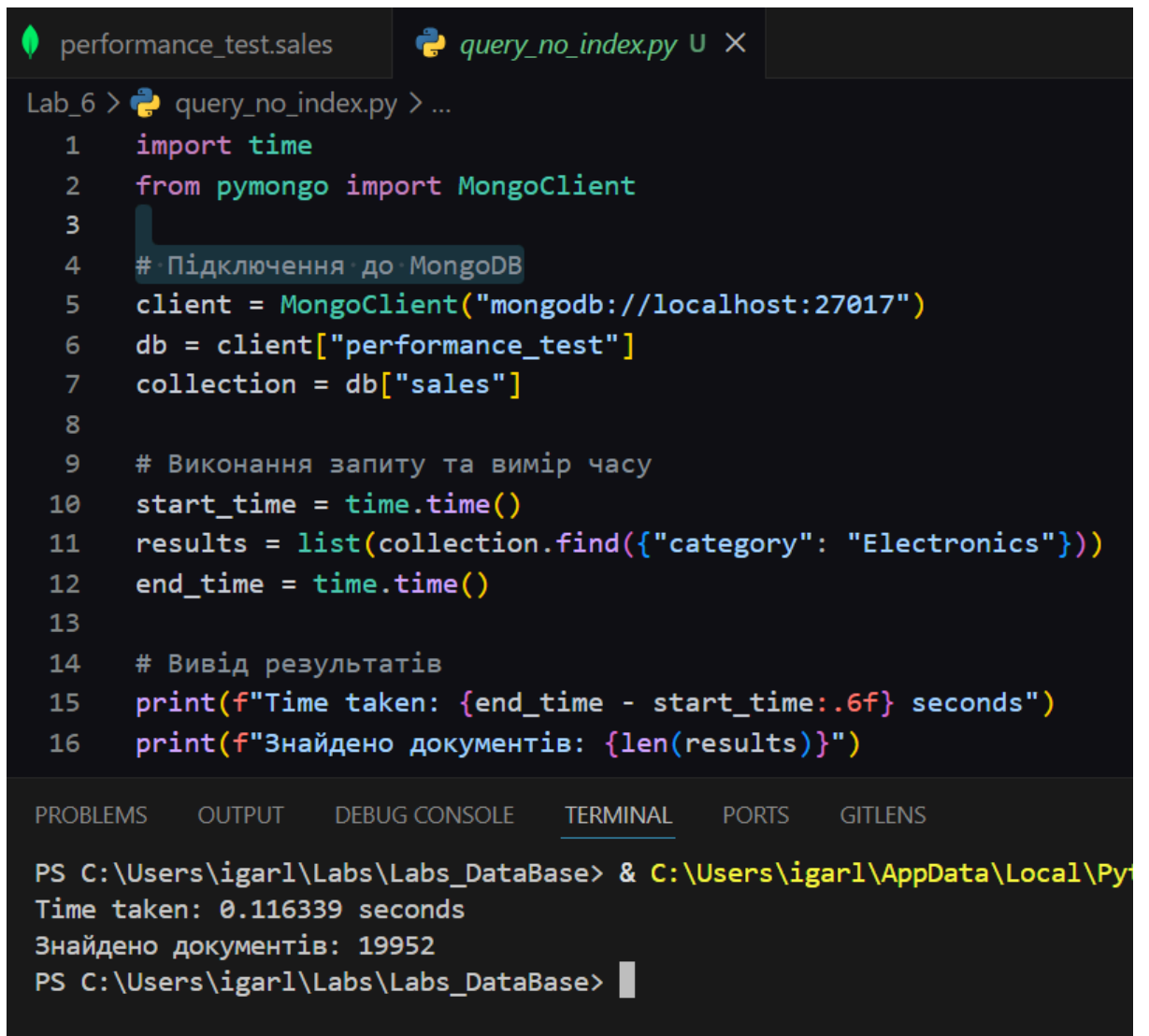
3. Оптимізація запитів за допомогою індексів

Виконати пошуковий запит без індексу та виміряти час виконання ([query_no_index.py](#)).

Створити індекс на полі [category](#) ([create_index.py](#)).

Повторно виконати запит та порівняти час виконання.

Дослідити складений індекс ([category](#), [timestamp](#)) та його вплив на продуктивність.



The image shows a code editor with a file named `query_no_index.py` open. The code is a Python script that connects to a MongoDB database, queries a collection named 'sales' for documents where the category is 'Electronics', and prints the time taken and the number of documents found. The script is executed in a terminal window, which shows the output: 'Time taken: 0.116339 seconds' and 'Знайдено документів: 19952'.

```
Lab_6 > query_no_index.py > ...
1  import time
2  from pymongo import MongoClient
3
4  # Підключення до MongoDB
5  client = MongoClient("mongodb://localhost:27017")
6  db = client["performance_test"]
7  collection = db["sales"]
8
9  # Виконання запиту та вимір часу
10 start_time = time.time()
11 results = list(collection.find({"category": "Electronics"}))
12 end_time = time.time()
13
14 # Вивід результатів
15 print(f"Time taken: {end_time - start_time:.6f} seconds")
16 print(f"Знайдено документів: {len(results)}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
PS C:\Users\igarl\Labs\Labs_DataBase> & C:\Users\igarl\AppData\Local\Py
Time taken: 0.116339 seconds
Знайдено документів: 19952
PS C:\Users\igarl\Labs\Labs_DataBase> █
```

Рис. 3. Виконання запиту без індексу , час виконання 0.116339 сек.

```
performance_test.sales create_index.py U X
Lab_6 > create_index.py > ...
1 from pymongo import MongoClient
2
3 # Підключення до MongoDB
4 client = MongoClient("mongodb://localhost:27017")
5 db = client["performance_test"]
6 collection = db["sales"]
7
8 # Створення індексу
9 collection.create_index("category")
10 print("Індекс створено")
```

Рис. 4. Код для створення індексу

```
PS C:\Users\igarl\Labs\Labs_DataBase> &
Time taken: 0.082984 seconds
Знайдено документів: 19952
PS C:\Users\igarl\Labs\Labs_DataBase>
```

Рис. 5. Час виконання з індексом 0.082984 с. , набагато швидше

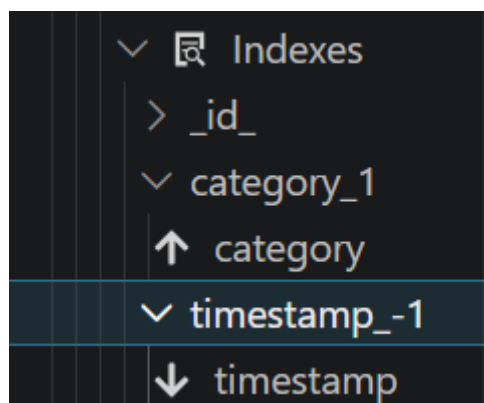


Рис. 6. Створено додаткові індекси.

```
PS C:\Users\igarl\Labs\Labs_DataBase> &
Time taken: 0.087798 seconds
Знайдено документів: 19952
PS C:\Users\igarl\Labs\Labs_DataBase>
```

Рис. 7. Час виконання сильно не змінився адже індекси вже були створені під час написання звіту.

Частина 2: Налаштування реплікації у MongoDB

1. Запустити три екземпляри MongoDB у режимі реплікації.

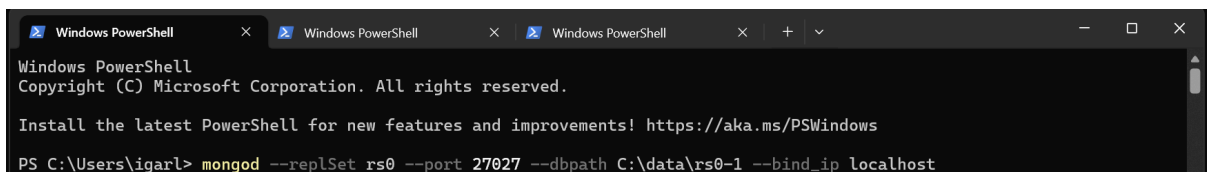


Рис. 8. В трьох окремих вікнах запущено БД розміщені в окремих папках

2. Ініціалізувати реплікаційний набір та додати вузли.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\igarl> mongosh --port 27027
Current Mongosh Log ID: 6a2a6b14ee9edc6ald7c2906
Connecting to:      mongodb://127.0.0.1:27027/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2
.7.0
Using MongoDB:      8.2.5
Using Mongosh:       2.7.0
mongosh 2.8.3 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-06-11T10:57:54.307+03:00: Access control is not enabled for the database. Read and write access to data and conf
iguration is unrestricted
-----

rs0 [direct: primary] test> rs.initiate()
MongoServerError[AlreadyInitialized]: already initialized
rs0 [direct: primary] test> |
```

Рис.9. Ініціалізація реплікації, я вже її ініціалізував раніше , тому зараз просто переконався, що вона працює

```
rs0 [direct: primary] test> rs.add("localhost:27028")
MongoServerError[NewReplicaSetConfigurationIncompatible]: Found two member configurations with same host field, members.
1.host == members.3.host == localhost:27028
rs0 [direct: primary] test> |
```

Рис.10. Тут я переконався, що додавав БД до кластера.

```

members: [
  {
    _id: 0,
    name: 'localhost:27027',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 351,
    optime: { ts: Timestamp({ t: 1781165021, i: 1 }), t: Long('3') },
    optimeDate: ISODate('2026-06-11T08:03:41.000Z'),
    optimeWritten: { ts: Timestamp({ t: 1781165021, i: 1 }), t: Long('3') },
    optimeWrittenDate: ISODate('2026-06-11T08:03:41.000Z'),
    lastAppliedWallTime: ISODate('2026-06-11T08:03:41.566Z'),
    lastDurableWallTime: ISODate('2026-06-11T08:03:41.566Z'),
    lastWrittenWallTime: ISODate('2026-06-11T08:03:41.566Z'),
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1781164689, i: 1 }),
    electionDate: ISODate('2026-06-11T07:58:09.000Z'),
    configVersion: 5,
    configTerm: 3,
    self: true,
    lastHeartbeatMessage: ''
  },

```

Рис. 11. Видно, що зараз lh27027 є PRIMARY

3. Виконати запис у Primary та перевірити доступність даних на Secondary-вузлах.

```

rs0 [direct: primary] test> use performance_test
switched to db performance_test
rs0 [direct: primary] performance_test> db.sales.insertOne({
|   "customer_id": 9999,
|   "category": "Electronics",
|   "amount": 1250.0,
|   "timestamp": new Date(),
|   "note": "Тестовий запис для реплікації"
| })
| {
|   acknowledged: true,
|   insertedId: ObjectId('6a2a6d68ee9edc6a1d7c2907')
| }
rs0 [direct: primary] performance_test> |

```

Рис. 12. Додаю тестовий документ


```
rs0 [direct: secondary] performance_test> db.sales.find({ "customer_id": 9999 })
[
  {
    _id: ObjectId('6a2a6d68ee9edc6a1d7c2907'),
    customer_id: 9999,
    category: 'Electronics',
    amount: 1250,
    timestamp: ISODate('2026-06-11T08:10:16.636Z'),
    note: 'Тестовий запис для реплікації'
  }
]
rs0 [direct: secondary] performance_test> |
```

Рис. 13. Тестовий документ знайшло на secondary вузлі, все працює.

4. Відключити Primary та дослідити поведінку системи.

```
{
  _id: 1,
  name: 'localhost:27028',
  health: 1,
  state: 1,
  stateStr: 'PRIMARY',
  uptime: 1101,
  optime: { ts: Timestamp({ t: 1781165777, i: 1 }), t: Long('4') },
  optimeDate: ISODate('2026-06-11T08:16:17.000Z'),
  optimeWritten: { ts: Timestamp({ t: 1781165777, i: 1 }), t: Long('4') },
  optimeWrittenDate: ISODate('2026-06-11T08:16:17.000Z'),
  lastAppliedWallTime: ISODate('2026-06-11T08:16:17.764Z'),
  lastDurableWallTime: ISODate('2026-06-11T08:16:17.764Z'),
  lastWrittenWallTime: ISODate('2026-06-11T08:16:17.764Z'),
  syncSourceHost: '',
  syncSourceId: -1,
  infoMessage: '',
  electionTime: Timestamp({ t: 1781165757, i: 1 }),
  electionDate: ISODate('2026-06-11T08:15:57.000Z'),
  configVersion: 5,
  configTerm: 4,
  self: true,
  lastHeartbeatMessage: ''
}
```

Рис. 14. Тепер вузол lh27028 взяв на себе роль primary вузла.

Частина 3: Реалізація шардінгу у MongoDB

1. Запустити два екземпляри MongoDB у режимі шардів.
2. Додати шарди через MongoDB Shell.

```
PS C:\Users\igarl> mongod --configsvr --replSet cfg --port 27030 --dbpath C:\data\cfg --bind_ip localhost
```

Рис. 15 . Запуск конфігураційного сервера

```

PS C:\Users\igarl> mongosh --port 27030
Current Mongosh Log ID: 6a2a6f5c2ed5eccdc47c2906
Connecting to:      mongodb://127.0.0.1:27030/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:      8.2.5
Using Mongosh:       2.7.0
mongosh 2.8.3 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-06-11T11:18:18.236+03:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

cfg [direct: primary] test> rs.initiate({ _id: "cfg", configsvr: true, members: [ { _id: 0, host: "localhost:27030" } ] })
MongoServerError[AlreadyInitialized]: already initialized
cfg [direct: primary] test> |

```

Рис. 16. Його уже ініціалізовано , я просто відтворюю свої попередні кроки.

```

PS C:\Users\igarl> mongod --shardsvr --replSet shard1rs --port 27021 --dbpath C:\data\shard1 --bind_ip localhost

```

Рис. 17. Запуск першого шарду.

```

PS C:\Users\igarl> mongosh --port 27021
Current Mongosh Log ID: 6a2a7046a86a3433ac7c2906
Connecting to:      mongodb://127.0.0.1:27021/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:      8.2.5
Using Mongosh:       2.7.0
mongosh 2.8.3 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-06-11T11:22:19.772+03:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

shard1rs [direct: primary] test> rs.initiate({ _id: "shard1rs", members: [ { _id: 0, host: "localhost:27021" } ] })
MongoServerError[AlreadyInitialized]: already initialized
shard1rs [direct: primary] test> |

```

Рис. 18. Вже ініціалізовано раніше.

```

PS C:\Users\igarl> mongod --shardsvr --replSet shard2rs --port 27022 --dbpath C:\data\shard2 --bind_ip localhost

```

Рис.19. Запуск другого шарду.

```

test> rs.initiate({ _id: "shard2rs", members: [ { _id: 0, host: "localhost:27022" } ] })
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1781166271, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1781166271, i: 1 })
}
shard2rs [direct: secondary] test> |

```

Рис. 20. Цей шард я ще не ініціалізував раніше

```

PS C:\Users\igarl> mongos --configdb cfg/localhost:27030 --port 27020 --bind_ip localhost

```

Рис. 21. Запуск mongos.

```
[direct: mongos] test> sh.addShard("shard1rs/localhost:27021")
| sh.addShard("shard2rs/localhost:27022")
| sh.enableSharding("performance_test")
| sh.shardCollection("performance_test.sales", { category: 1 })
{
  collectionssharded: 'performance_test.sales',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1781166492, i: 170 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1781166492, i: 169 })
}
[direct: mongos] test> |
```

Рис. 22. Додаю шарди та вмикаю шардинг

The screenshot shows the MongoDB Compass interface. At the top, the database 'performance_test' and collection 'sales' are selected. The 'Aggregations' tab is active, showing a single stage named '\$group'. The aggregation pipeline is defined as:

```
1 {
2   _id: "$category",
3   count: {
4     $sum: 1
5   }
6 }
```

The 'Preview of documents' section shows four sample documents from the 'sales' collection, each with a unique '_id', a 'customer_id', a 'category' (either 'Books' or 'Sports'), an 'amount', and a 'timestamp'. The 'Output preview after \$group stage' section shows the result of the aggregation, which is a list of documents where each document represents a category and its total count. For example, the 'Home' category has a count of 19928, and the 'Sports' category has a count of 19745.

Рис. 23. Виконання агрегаційного запиту {

```
_id: "$category",
count: {
  $sum: 1
}
}
```



Рис. 24.Схема згенерованих даних , на ній видно що деякі категорії даних генерувалися частіше ніж інші на 5-10%, але це не критично.

Висновки

Під час виконання лабораторної роботи було успішно досліджено архітектурні можливості та методи оптимізації бази даних MongoDB. Практично доведено, що використання правильних індексів кардинально зменшує час виконання пошукових запитів. Було успішно розгорнуто реплікаційний набір, який продемонстрував механізм забезпечення відмовостійкості бази даних та автоматичного відновлення після падіння головного вузла. Також налаштовано шардований кластер із маршрутизатором, що підтвердило можливість горизонтального масштабування системи для роботи з великими навантаженнями. Окрім того, засвоєно використання Aggregation Pipeline для ефективної аналітики та візуалізації даних.

